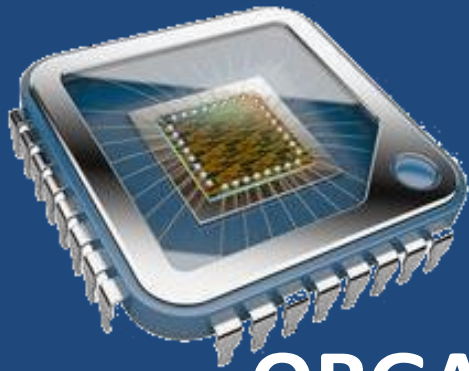
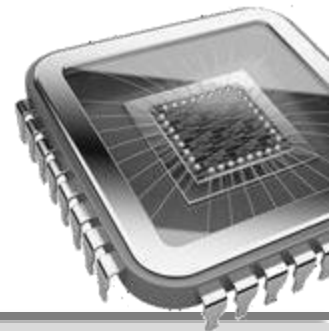




ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES

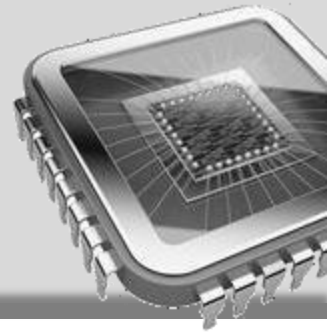
AULA 02: FUNCIONAMENTO DE UM COMPUTADOR





O QUE É UM COMPUTADOR?

COMPUTADOR

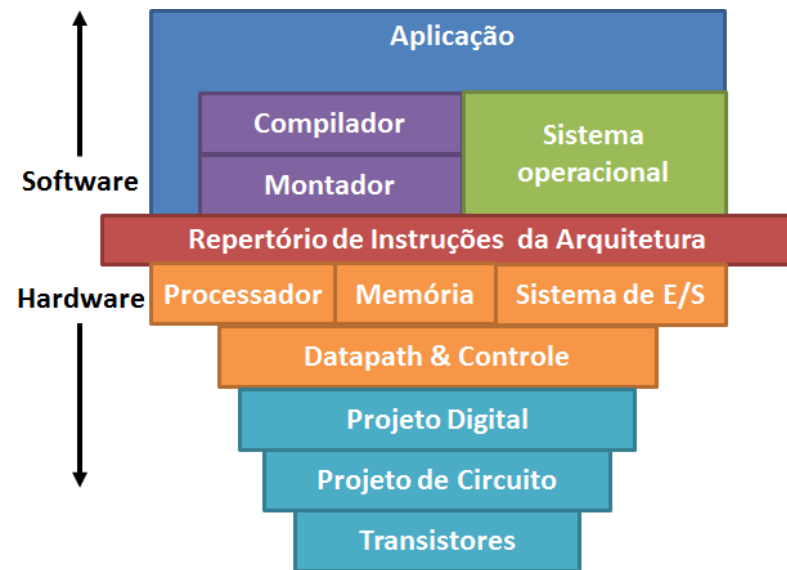


- Uma máquina digital que pode resolver problemas executando um programa (software), além do Sistema Operacional que dever existir.

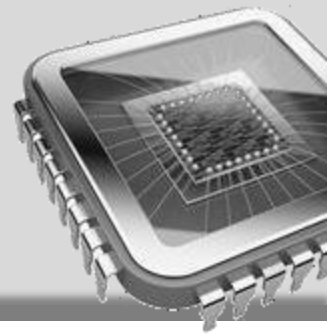
Software é um conjunto de instruções que descrevem a maneira de realizar determinada tarefa.

Hardware é responsável por reconhecer e executar o conjunto de instruções.

Sistema operacional é um software, ou conjunto de softwares, cuja função é administrar e gerenciar os recursos de um sistema, desde componentes de hardware e sistemas de arquivos a programas de terceiros, estabelecendo a interface entre o computador e o usuário.

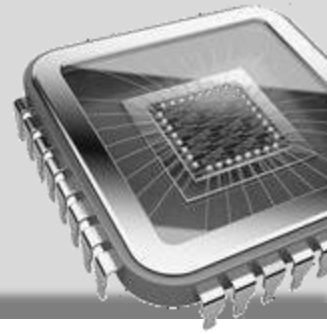


COMPILADOR



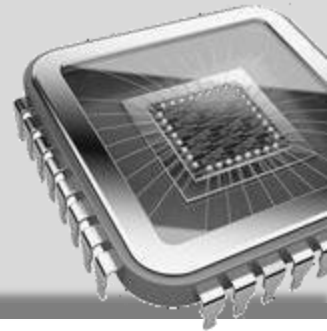
- O compilador é um programa que lê e analisa o código fonte da aplicação, ou seja, o código que nós escrevemos, e gera a partir dele um que pode ser executado.
- Normalmente os compiladores são programas com uma lógica complexa, já que se utilizam de muitas técnicas para acelerar a aplicação e reduzir os requisitos do sistema onde essa aplicação irá ser executada.

MONTADOR



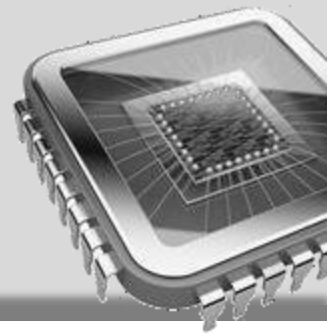
- O assembler é um programa que traduz código assembly para **linguagem de máquina**. O código assembly é outra linguagem de programação semelhante à linguagem de máquina, mas é mais fácil de ser entendida e escrita por humanos.
- O processo de montagem inclui uma única etapa, a geração de código. O resultado da montagem é um arquivo objeto que pode ser vinculado a outras bibliotecas para produzir um arquivo executável.

LINGUAGEM DE MÁQUINA



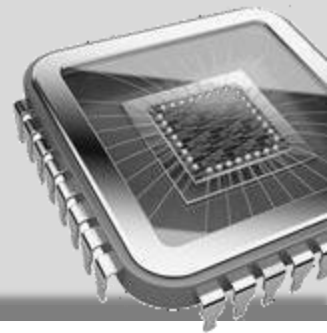
- Linguagem de máquina é a linguagem que a maioria do hardware informático compreende.
- É a linguagem de nível mais baixo que pode ser usada para controlar o comportamento de um computador.
- É uma sequência de dígitos binários (um e zeros) que são interpretados pela CPU para realizar operações.
- Um programa escrito em linguagem de máquina é chamado de código de máquina.

TIPOS DE LINGUAGEM DE MÁQUINA



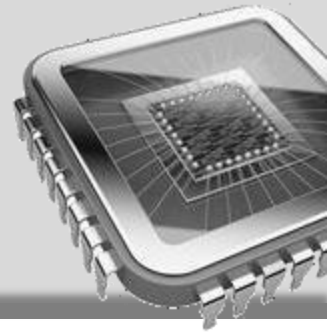
- Existem dois tipos de linguagem de máquina: nativa e interpretada.
- Linguagem de máquina nativa é a linguagem que é nativamente compreendida pelo hardware do computador.
- É normalmente escrita em binário e é específica para o tipo particular de hardware.
- A linguagem de máquina interpretada é uma língua de nível superior que é convertida em linguagem de máquina nativa utilizando um intérprete ou INTERPRETADOR.

ASSEMBLY & ASSEMBLER



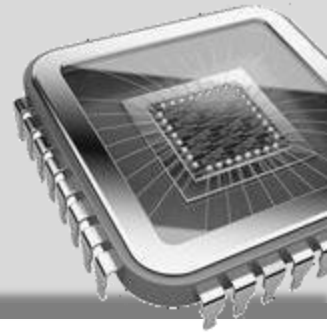
- Assembly é chamada de linguagem de baixo nível. Isso porque está diretamente ligada às ordens da máquina. Cerca de grande parte dos programadores interessados em conhecer as “entranhas” dos computadores, escolhem estudar Assembly.
- Eles viam melhorias no desempenho, segurança e entendimento de hardware.
- Isso significa que você pode controlar diretamente o computador. Ela usa códigos parecidos com as instruções do processador.
- A cada tipo de computador, há uma **Linguagem Assembly** específica. Por exemplo, há uma para processadores Intel. Com ela, é possível fazer programas que controlam em detalhes o computador.

ASSEMBLER



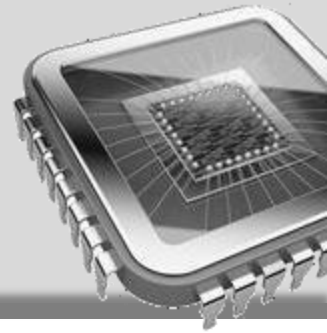
- Assembler, como o próprio nome diz é um **Montador**, e **não um Compilador**, ainda que ele funcione de forma muito semelhante.
- Ele pega um texto que é um código de programação e transforma em código binário (código de máquina).
- O que difere de um compilador é justamente que as instruções na linguagem possuem uma relação um para um no código binário.

MONTADOR & COMPILADOR



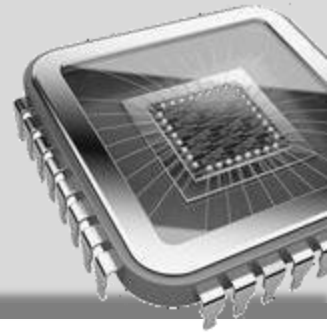
- A principal diferença entre um **Montador** e um **Compilador** é que o **Montador** traduz o código assembly para a linguagem de máquina, enquanto o **Compilador** traduz o código de uma linguagem de alto nível para a linguagem de máquina.
- Ambos têm suas vantagens e desvantagens, e a escolha de um ou outro dependerá do projeto e dos objetivos do programador.

Assembly: a linguagem



- Assembly é uma linguagem de programação de baixo nível composta por *mnemônicos* — instruções legíveis por humanos que correspondem diretamente às instruções de máquina de um processador.
- **Para que serve:** Permite ao programador controlar diretamente o hardware, manipulando registradores, memória e fluxo de execução.
- **Exemplo:** MOV AX, 1 é uma instrução em Assembly que move o valor 1 para o registrador AX.
- **Características:**
 - Extremamente próxima do hardware.
 - Cada arquitetura (x86, ARM, MIPS etc.) tem seu próprio "dialetto" de Assembly.
 - Usada quando se precisa de desempenho máximo ou controle absoluto sobre o sistema.

Assembly: a linguagem



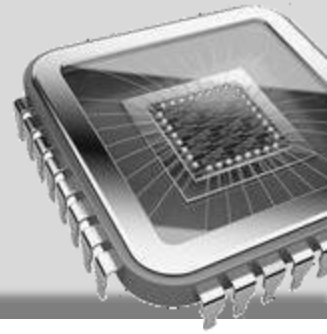
Assembler: o montador

O que é: Assembler é o programa que *traduz* o código escrito em Assembly para linguagem de máquina (binário), que o processador realmente entende.

Função: Atua como um "montador", convertendo os mnemônicos em instruções binárias específicas para o processador.

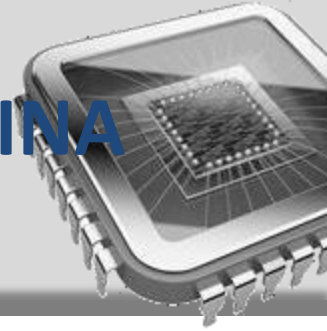
Diferença de um compilador: Enquanto compiladores traduzem linguagens de alto nível (como C ou Java), o assembler faz uma tradução direta e simples, geralmente instrução por instrução.

VANTAGENS DA LINGUAGEM DE MÁQUINA



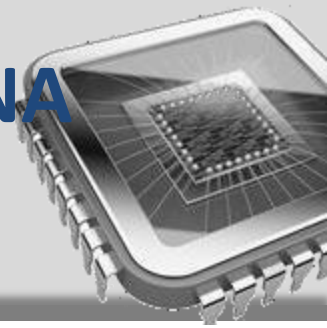
- Uma das maiores vantagens da linguagem de máquina é que é a forma mais eficiente de controlar o hardware do computador.
- É mais rápido e mais eficiente do que as línguas de alto nível e a linguagem de montagem.
- É também mais fácil de depurar, uma vez que os erros podem ser identificados e corrigidos rapidamente.

DESVANTAGENS DA LINGUAGEM DE MÁQUINA



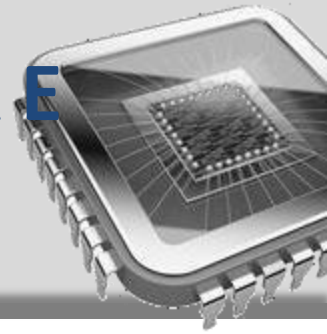
- A maior desvantagem da linguagem de máquina é que é difícil de escrever e compreender.
- É também muito difícil de modificar, uma vez que quaisquer alterações requerem que todo o programa seja reescrito.

RELAÇÃO ENTRE A LINGUAGEM DE MÁQUINA E A LINGUAGEM DE MONTAGEM



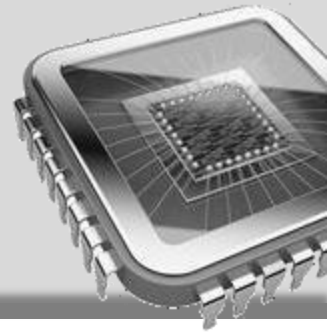
- A linguagem de montagem é uma linguagem de nível superior que é convertida em linguagem de máquina nativa utilizando um assembler.
- A linguagem de montagem é mais fácil de compreender e modificar do que a linguagem de máquina, mas continua a ser muito mais baixa do que as línguas de alto nível.

RELAÇÃO ENTRE LINGUAGEM DE MÁQUINA E LINGUAGEM DE ALTO NÍVEL

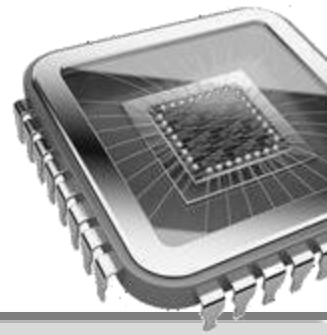


- As línguas de alto nível são mais fáceis de escrever e compreender do que a linguagem de máquina, mas têm de ser convertidas em linguagem de máquina nativa para poderem ser executadas.
- Isto é normalmente feito utilizando um **compilador** ou um **intérprete**.

FUTURO DA LINGUAGEM DE MÁQUINA

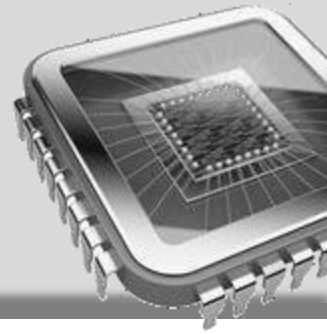


- Devido à sua eficiência, a linguagem de máquina continuará a ser usada no futuro.
- No entanto, é provável que se torne menos importante à medida que as línguas de alto nível se tornem mais avançadas e mais fáceis de utilizar.
- Será também utilizada em combinação com outras línguas, como a linguagem de montagem, para criar programas mais eficientes e poderosos.

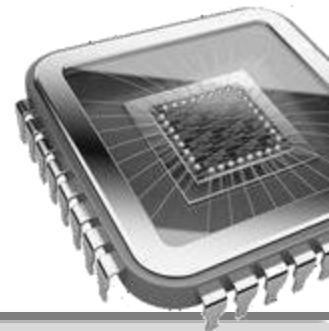


COMO UM PROGRAMA ESCRITO EM ALTO NÍVEL É ENTENDIDO E EXECUTADO PELO HARDWARE?

LINGUAGEM DO HARDWARE

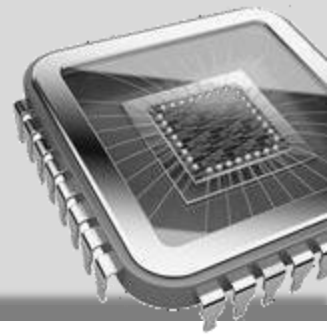


- O hardware só entende sinais elétricos (dois valores)
 - Ligado (*on*)/Desligado (*off*)
 - Zeros (0) e uns (1) (números binários)
- As instruções para o computador são sequências de números binários (linguagem de máquina)
- A linguagem de máquina está muito distante de uma linguagem natural (linguagem humana)
- Programar em linguagem de máquina é **impraticável!**



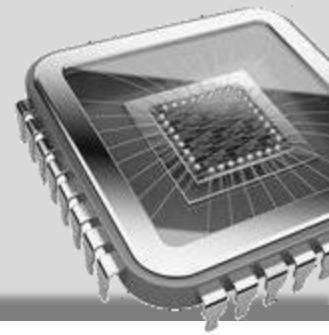
COMO INSTRUIR OS COMPUTADORES?

SISTEMA HIERÁRQUICO

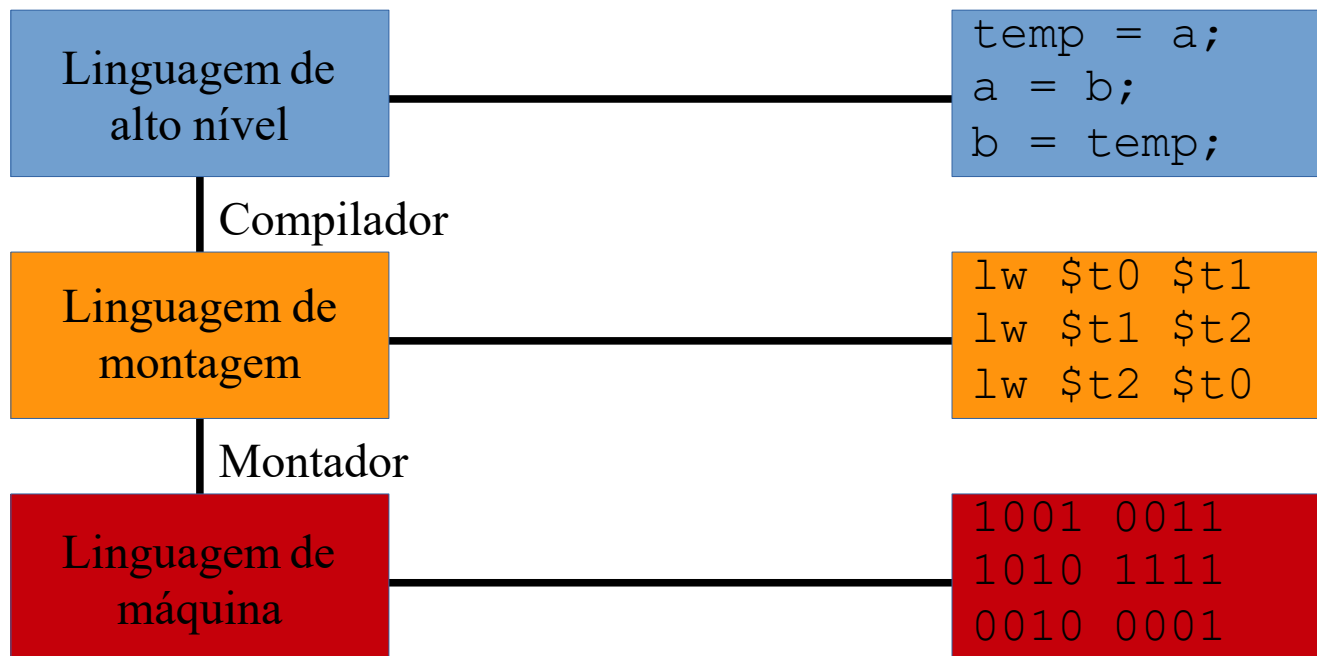


- Criar uma **organização estruturada de computadores** para facilitar a comunicação **homem-computador**.
 - Criando uma hierarquia de abstrações de níveis mais alto baseados nos níveis mais baixos.
 - Um conjunto de subsistemas inter-relacionados, onde, cada qual, possui uma estrutura hierárquica, que contém em seu nível mais baixo subsistemas elementares.
 - Cada nível depende apenas de uma caracterização abstrata e simplificada do sistema de nível imediatamente inferior.
 - Abstraindo os conceitos de hardware.
 - Criando linguagens de programação.
 - Linguagens entendidas pelos seres humanos e que possam ser traduzidas para a linguagem de máquina (entendida pelos computadores)

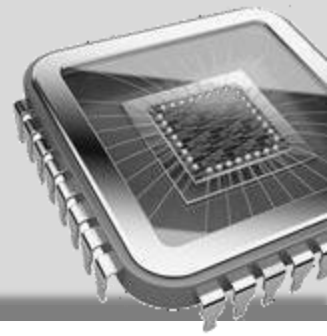
SISTEMAS HIERÁRQUICO



- As linguagens de programação variam de acordo com o seu nível de abstração.

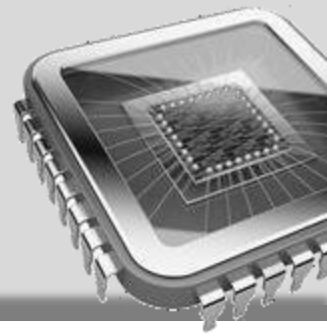


Linguagem Assembly



- Compreender a relação entre Assembly e arquitetura de processadores
- Utilizar registradores
- Executar instruções básicas:
 - MOV
 - ADD
 - SUB
 - INC
- Exibir resultados simples na tela
- Interpretar cada linha de código Assembly

Linguagem Assembly



- Registradores principais

Principais registradores

◆ 1. RAX — Acumulador

```
</> assembly
```

```
RAX (64 bits)
```

```
EAX (32 bits)
```

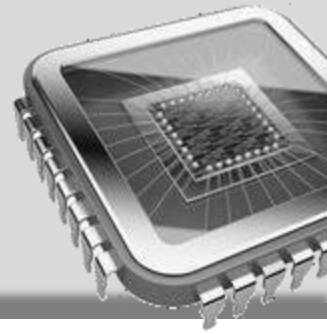
```
AX (16 bits)
```

```
AL (8 bits)
```

Função:

- Principal registrador para cálculos
- Muito usado em operações matemáticas

Linguagem Assembly



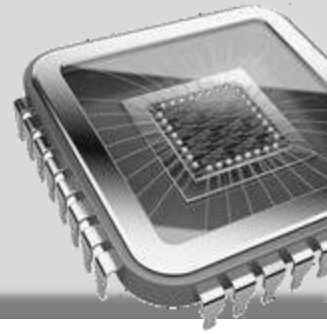
Ferramenta Online Recomendada

Os alunos podem usar: **<https://www.onlinegdb.com>**

Configuração:

Assembler: NASM

Linguagem Assembly



Estrutura Básica de um Programa Assembly

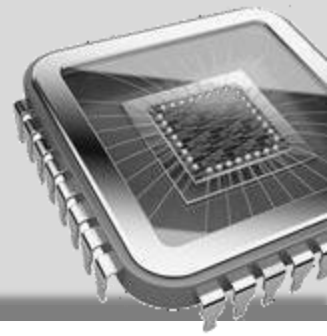
Todo programa Assembly possui três partes:

section .data

section .bss

section .text

Linguagem Assembly



- **section .data**

Guarda dados inicializados

Exemplo:

```
num1 db 5
```

section .bss

Guarda variáveis sem valor inicial

Exemplo:

```
resultado resb 1
```

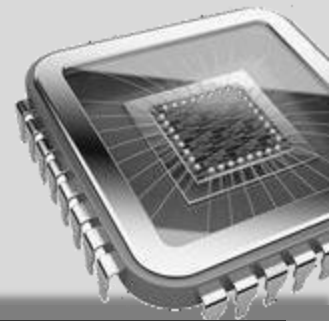
section .text

Onde ficam as instruções do programa

Exemplo:

```
global _start
```

```
_start:
```



Exemplo 1 –
Soma
mostrando
resultado na
tela

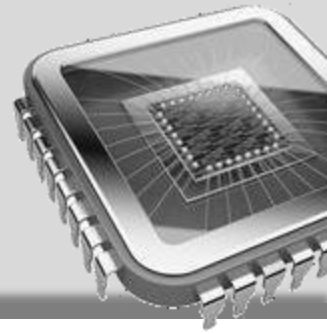
```
1  section .data
2
3  num1 db 3
4  num2 db 4
5
6  msg db "Resultado: "
7  tam equ $-msg
8
9  section .bss
10
11  res resb 1
12
13  section .text
14
15  global _start
16
17  _start:
18
19      mov al, [num1]
20      add al, [num2]
21
```

```
21
22      add al, 48
23      mov [res], al
24
25      mov eax, 4
26      mov ebx, 1
27      mov ecx, msg
28      mov edx, tam
29      int 0x80
30
31      mov eax, 4
32      mov ebx, 1
33      mov ecx, res
34      mov edx, 1
35      int 0x80
```



Resultado: 7

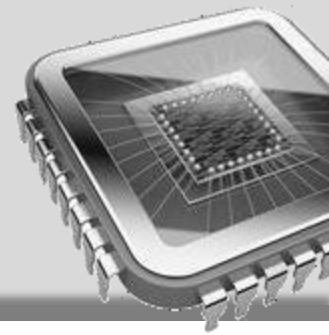
1. Estrutura geral do programa



Um programa Assembly normalmente é dividido em três áreas:

Seção	Função
<code>.data</code>	dados inicializados
<code>.bss</code>	variáveis reservadas
<code>.text</code>	código executável

1.1 – Seção .data



```
section .data
```

Essa parte do programa define **dados que já possuem valores iniciais**.

Definição das variáveis

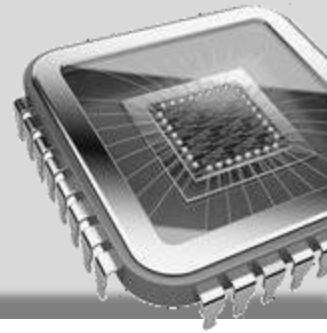
num1 db 3

Então na memória teremos:
num1 = 3

Significa:

Parte	Significado
num1	nome da variável
db	define byte
3	valor armazenado

1.1 – Seção .data



Cria outra variável.

num2 db 4

Então na memória teremos:
num2 = 4

Mensagem que será exibida

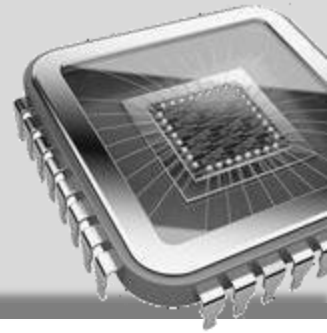
msg db "Resultado: "

Cria um **texto armazenado na memória**.
Na memória ficará algo como:

Resultado:

Cada caractere é armazenado em **ASCII**.

1.1 – Seção .data



Cálculo do tamanho da mensagem

tam equ \$-msg

Aqui acontece algo interessante.

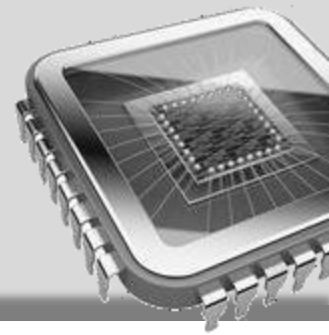
Parte	Significado
\$	posição atual na memória
msg	início da string

Então:

tam = tamanho da string

No caso: Resultado: -> 11 caracteres

1.2 – Seção .bss



section .bss

Essa área guarda **variáveis** que ainda não têm valor definido.

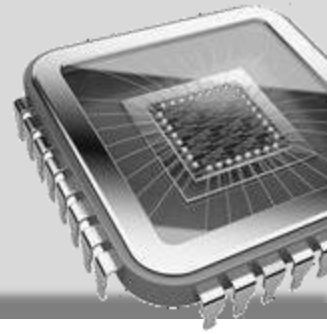
Reserva de memória --→ **res resb 1**

Significa:

Parte	Função
res	nome da variável
resb	reserve byte
1	reservar 1 byte

Então estamos reservando espaço para **guardar o resultado da soma**.

1.2 – Seção .text



section .text

Aqui fica **o código que será executado pela CPU.**

- Ponto inicial do programa

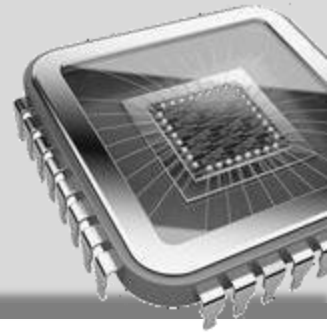
global _start

Informa ao sistema operacional onde começa o programa.

_start:

Label (marcador) indicando o **início da execução.**

1.2 – Seção .text



- Primeiro comando
mov al, [num1]

Significado:

AL = valor de num1

- Depois da execução:
AL = 3

Por que usar colchetes [] ?

- **Com colchetes**

mov al, [num1]

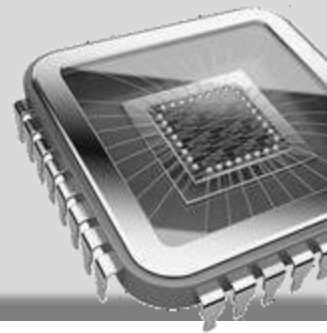
pegar o VALOR armazenado na memória

- **Sem colchetes**

mov al, num1

pegar o ENDEREÇO da variável

1.2 – Seção .text



Exemplo:

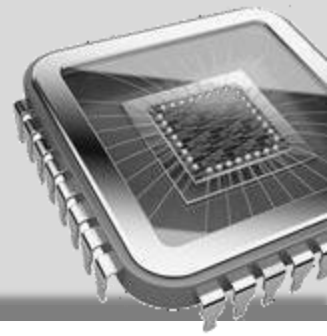
Variável	Endereço	Valor
num1	1000	3

```
[num1] = 3  
num1 = 1000
```

Então:

Código	Resultado
<code>mov al,[num1]</code>	AL = 3
<code>mov al,num1</code>	AL = endereço

1.2 – Seção .text



- Soma

add al, [num2]

Significa:

$AL = AL + num2$

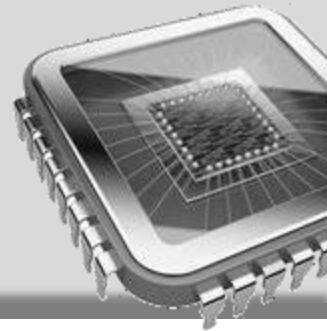
Antes:

$AL = 3$

Depois:

$AL = 7$

1.2 – Seção .text



- Conversão para ASCII
add al, 48

Número	ASCII
0	48
1	49
2	50
3	51
4	52
5	53
6	54
7	55

Por que isso é necessário?

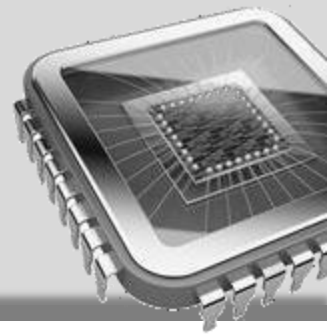
Porque o computador guarda números **em binário**, mas para mostrar na tela precisamos de **ASCII**.

Então:

$$7 + 48 = 55$$

ASCII 55 = caractere '7'

1.2 – Seção .text



- Guardando o resultado

mov [res], al Significa → res = AL

Agora a memória contém: res = '7'

- Preparando impressão na tela

mov eax,4

```
mov eax,4    ; código da função write
int 0x80
```

Etapa

Ação

mov eax,4

escolhe a função (write)

int 0x80

chama o kernel

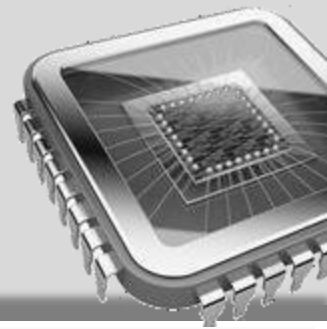
kernel

escreve na tela

Você está dizendo:

Kernel, execute a função WRITE

1.2 – Seção .text



```
mov ebx,1
```

Define o destino da saída.

```
1 = tela (stdout)
```

```
mov ebx,1
```

Define o destino da saída.

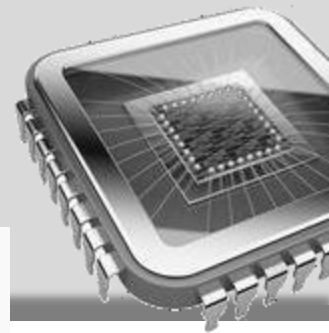
```
1 = tela (stdout)
```

```
</> assembly
```

```
mov ecx,msg
```

Define o endereço da mensagem.

1.2 – Seção .text



```
int 0x80
```

Significa:

```
interrupção de software
```

É um mecanismo usado para **pedir serviços ao sistema operacional**.

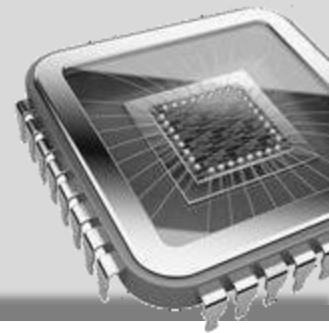
Fluxo:

```
programa → kernel Linux → hardware
```

Neste caso estamos pedindo:

```
escrever texto na tela
```

1.2 – Seção .text



Estrutura da chamada

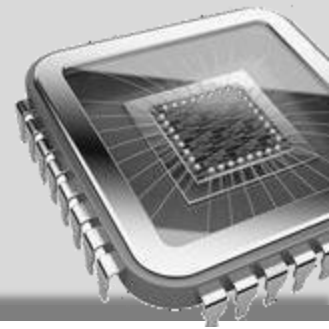
Antes do `int 0x80` colocamos parâmetros:

Registrador	Função
EAX	número da syscall
EBX	argumento 1
ECX	argumento 2
EDX	argumento 3

Para `write`:

```
eax = 4
ebx = destino
ecx = endereço
edx = tamanho
```

1.2 – Seção .text



Imprimindo o resultado

Depois o programa executa novamente:

Depois o programa executa novamente:

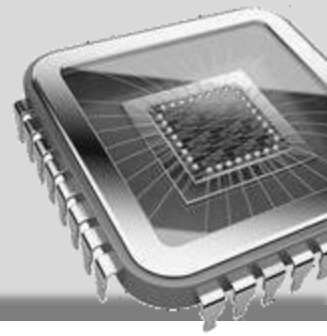
```
</> assembly  
  
mov eax,4  
mov ebx,1  
mov ecx,res  
mov edx,1  
int 0x80
```

Agora o sistema imprime o caractere armazenado em `res`.

Saída final:

Resultado: 7

1.2 – Seção .text



Encerrando o programa

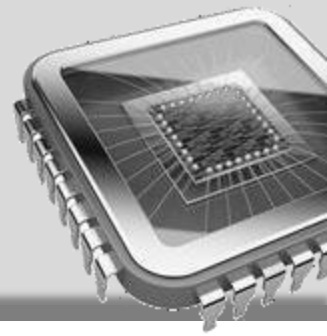
```
mov eax,1
```

```
int 0x80
```

Aqui usamos a syscall:

```
1 = exit
```

Isso termina o programa.



Exercício

Modificar os valores para calcular:

Exercício 1

$$2 + 6$$

Exercício 2

$$(4 + 3) - 2$$